

Министерство науки и высшего образования Российской Федерации

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО

Приоритетный национальный проект «Образование»
Национальный исследовательский университет

И. В. ШОШМИНА

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ К
КУРСОВОЙ РАБОТЕ ПО
МАТЕМАТИЧЕСКОЙ ЛОГИКЕ
«ИСПОЛЬЗОВАНИЕ БИНАРНЫХ
РЕШАЮЩИХ ДИАГРАММ ДЛЯ
РЕШЕНИЯ ЛОГИЧЕСКИХ ЗАДАЧ»
(ЧЕРНОВИК)

Санкт-Петербург
Издательство Политехнического университета
2024

УДК 004.4
ББК 32.973.26 – 018.2
К26

Рецензенты:
Доктор технических наук,
Доктор технических наук, профессор
Санкт-Петербургского политехнического
университета

Шошмина И. В. Методические рекомендации к курсовой работе по математической логике «Использование бинарных решающих диаграмм для решения логических задач» (Черновик): учеб. пособие / И. В. Шошмина. — СПб. : Изд-во Политехн. ун-та, 2024. — 38 с.

ISBN

В пособии разбирается курсовая работа «Использование бинарных решающих диаграмм для решения логических задач» по дисциплине «Математическая логика». Поскольку бинарные решающие диаграммы (BDD) требуют элементарных знаний по математической логике, то первая часть пособия посвящена различным формам представления булевых функций, включая BDD, также в этой части разобраны основные алгоритмы построения и редукции BDD. Во второй части рассмотрены вопросы применения BDD при программировании на примере библиотеки BUDDy. В последней части сформулирована курсовая работа, рассмотрено решение упрощенного варианта курсовой работы и приведены индивидуальные варианты курсовых работ для студентов.

Учебное пособие предназначено для студентов вузов, обучающихся по направлениям подготовки бакалавров «Информатика и вычислительная техника», «Программная инженерия», «Фундаментальная информатика и информационные технологии», а также при обучении в системах повышения квалификации, в учреждениях дополнительного профессионального образования.

Работа выполнена в рамках реализации программы развития национального исследовательского университета «Модернизация и развитие политехнического университета как университета нового типа, интегрирующего мультидисциплинарные научные исследования и надотраслевые технологии мирового уровня с целью повышения конкурентоспособности национальной экономики»

Печатается по решению редакционно-издательского совета
Санкт-Петербургского государственного политехнического университета.

ISBN

© Шошмина И. В., 2024
© Санкт-Петербургский политехнический
университет Петра Великого, 2024

Оглавление

Введение	4
1. Представление булевых функций в виде бинарных решающих диаграмм	5
2. Реализация BDD в библиотеке BuDDy	12
3. Логические и их решение с помощью булевых функций	20
4. Курсовая работа «Использование бинарных решающих диаграмм для решения логических задач»	25
4.1. Пример решения упрощенного задания курсовой работы .	25
4.2. Условие курсовой работы	29
4.2.1. Изменение позиционирования объектов	29
4.2.2. Введение соседских отношений	30
4.2.3. Введение склейки	30
4.2.4. Творческая модификация	31
4.2.5. Формулировка условия работы	32
4.3. Индивидуальные варианты для курсовой работы	33
4.4. Требования к выполнению курсовой работы	34
Заключение	36
Библиографический список	37

ВВЕДЕНИЕ

В пособии рассматривается задача поиска всех возможных решений булевой функции. Такая задача достаточно часто встречается в так называемых «логических» задачах, т.е. в задачах, в которых задано несколько ограничений и решение находится в области, где все ограничения истинны. Решение такой задачи в общем случае экспоненциально зависит от количества двоичных переменных, необходимых для ее формализации. Большинство из традиционных форм представления булевой функции не позволяет решать такую задачу именно из-за экспоненциального размера.

В пособии рассматривается эффективная каноническая форма представления булевых функций — упорядоченная редуцированная бинарная решающая диаграмма (BDD). Отличительной ее особенностью от других форм представления булевых функций является то, что для многих практических функций она имеет полиномиальный, а иногда линейный размер. Для программирования задач с использованием BDD разработаны специальные библиотеки. Одна из таких библиотек BUDDy разбирается в пособии.

В курсовой работе с использованием BDD рассматривается хорошо известная логическая задача Эйнштена. В задаче задаются ограничения, формализовав которые можно найти решение задачи как истинную интерпретацию булевой функции. Решение задачи Эйнштейна формализовано и подробно рассмотрено в пособии. Для курсовой используется модификация задачи Эйнштейна, студент должен сам по индивидуальному варианту построить измененную формализацию, программу и найти решение, а также построить интерпретацию на естественном языке.

1. ПРЕДСТАВЛЕНИЕ БУЛЕВЫХ ФУНКЦИЙ В ВИДЕ БИНАРНЫХ РЕШАЮЩИХ ДИАГРАММ

Булевы функции отображают кортеж из n двоичных значений на значение 0 (False - ложно) или 1 (True - истинно). Кортеж из n двоичных значений называют интерпретацией (интерпретацией n двоичных переменных).

Основные задачи, связанные с булевыми функциями (БФ):

- найти значение БФ на конкретной интерпретации;
- найти все интерпретации, на которых функция истинна;
- проверить выполнимость, общезначимость, невыполнимость БФ;
- проверить на совпадение две БФ.

В зависимости от решаемых задач булева функция может быть задана разными способами. На табл. 1.1 булева функция f от трёх двоичных переменных p , q и r задана с помощью *таблицы истинности*. В каждой строке таблицы определены значения переменных и соответствующее им значение функции. На рис. 1.1 эта же функция f задана с помощью *семантического дерева*. Семантическое дерево — двоичное дерево. Узлы дерева помечены переменными. Сплошная дуга, исходящая из вершины, соответствует единичному значению переменной; пунктирная - нулевому значению переменной. В листьях дерева находятся значения функции. Таблица истинности и семантическое дерево являются *каноническими представлениями* булевой функции, т.е. они задают булеву функцию однозначно с точностью до порядка переменных. В таблице истинности и семантическом дереве, фактически, перечисляются значения функции для всех наборов переменных, что делает такое задание громоздким.

К более компактным формам представления булевой функции относятся аналитические формы, т.е. заданные с помощью формул. При построении булевых функций используются связки: $\neg$ — отрицание, $\vee$ — дизъюнкция, $\wedge$ — конъюнкция, $\oplus$ — исключающее «или», $\leftrightarrow$ — эквивалентность, $\rightarrow$ — импликация. Другие связки и их семантические значения можно найти, например, в [1].

С помощью суперпозиции этих связок можно построить формулу, задающую любую булеву функцию. Например, булеву функцию f из табл. 1.1 можно задать так: $f(p, q, r) = \neg p \vee q \oplus r \wedge q \wedge (p \vee r)$.

– Таблица 1.1 Таблица истинности булевой функции f

p	q	r	f
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

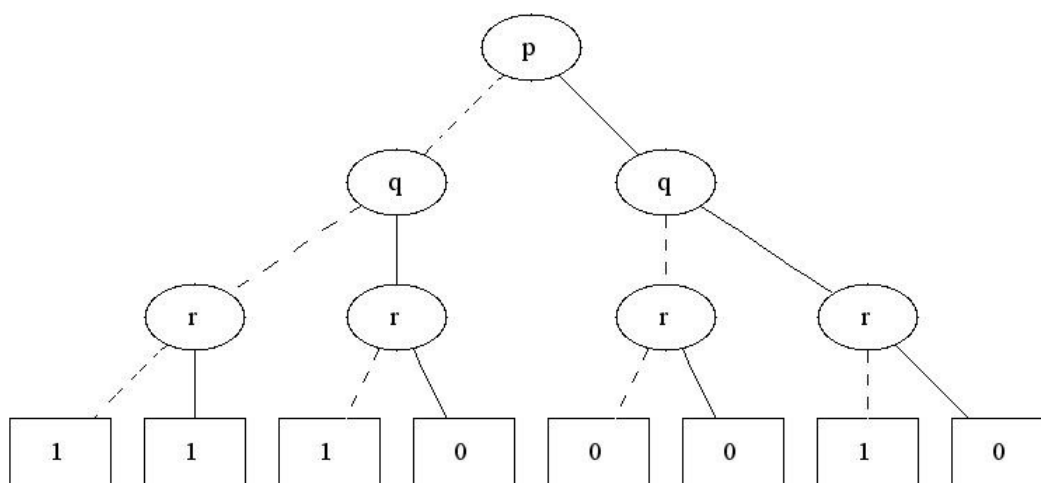


Рис. 1.1. Семантическое дерево булевой функции f

Аналитические формы представления булевой функции разнообразны. Они отличаются тем с помощью каких связок строится представление функции. В формуле выше никаких ограничений на выбор связок не накладывался. Одна и та же функция может быть задана с помощью бесконечного количества формул с произвольными связками.

Если аналитическая форма представляет из себя дизъюнкцию элементарных конъюнкций, т.е. конъюнкцию переменных с отрицанием или без (переменные в одной конъюнкции не повторяются), то такая форма называется *дизъюнктивной нормальной формой (ДНФ)*: $f(p, q, r) = \neg p \wedge \neg q \vee q \wedge \neg r$. Конъюнкция элементарных дизъюнкций определяет *конъюнктивную нормальную форму (КНФ)*: $f(p, q, r) = (\neg p \vee q) \wedge (\neg q \vee \neg r)$. Одну и ту же булеву функцию можно задать с помощью разных ДНФ и КНФ. С помощью этих форм трудно

определять, являются ли функции равносильными, т.е. совпадают ли они для любых наборов значений аргументов.

Совершенная дизъюнктивная нормальная форма (СДНФ) — это ДНФ, в которой каждая элементарная конъюнкция содержит все переменные:

$$f(p, q, r) = \neg p \wedge \neg q \wedge r \vee \neg p \wedge \neg q \wedge \neg r \vee p \wedge q \wedge \neg r \vee \neg p \wedge q \wedge \neg r.$$

Аналогично задается *совершенная конъюнктивная нормальная форма (СКНФ)*:

$$f(p, q, r) = (\neg p \vee q \vee \neg r) \wedge (\neg p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r) \wedge (p \vee \neg q \vee \neg r).$$

Если аналитическая форма ограничена связками $\oplus, \wedge$ и константой 1, то она называется *полиномом Жегалкина*: $f(p, q, r) = 1 \oplus p \oplus p \wedge q \oplus q \wedge r$.

От одной формы представления можно переходить к другой. С алгоритмами преобразований можно познакомиться, например, в [1, 4, 5].

СДНФ, СКНФ, полином Жегалкина являются каноническими представлениями булевой функции. Это означает, что сравнивать две функции между собой можно, сравнивая две строки. Однако в общем случае размер этих представлений полиномиально зависит от количества переменных булевой функции.

Все приведённые представления громоздки, поэтому с помощью этих представлений затруднительно решать задачи с количеством переменных больше 5-6.

В 1986 г. Randal Bryant предложил новую очень эффективную форму представления БФ, которую назвал бинарной решающей диаграммой – Binary Decision Diagrams (BDD) [1].

BDD — ациклический орграф, в котором отсутствуют повторения в структуре, с одной корневой вершиной, двумя листьями, помеченными 0 и 1, и промежуточными вершинами. Корневые и промежуточные вершины помечены переменными. Из каждой вершины выходит ровно два ребра. Пунктирной линией изображается дуга, соответствующая нулевому значению переменной, а сплошной — единичному. Листья обозначают значения булевой функции. На рис. 1.2а изображена BDD булевой функции, значение которой при $p = 0, q = 1, r = 0$ равно 1.

BDD отличает компактность представления. Сложность представления БФ в BDD пропорциональна числу вершин. Как правило,

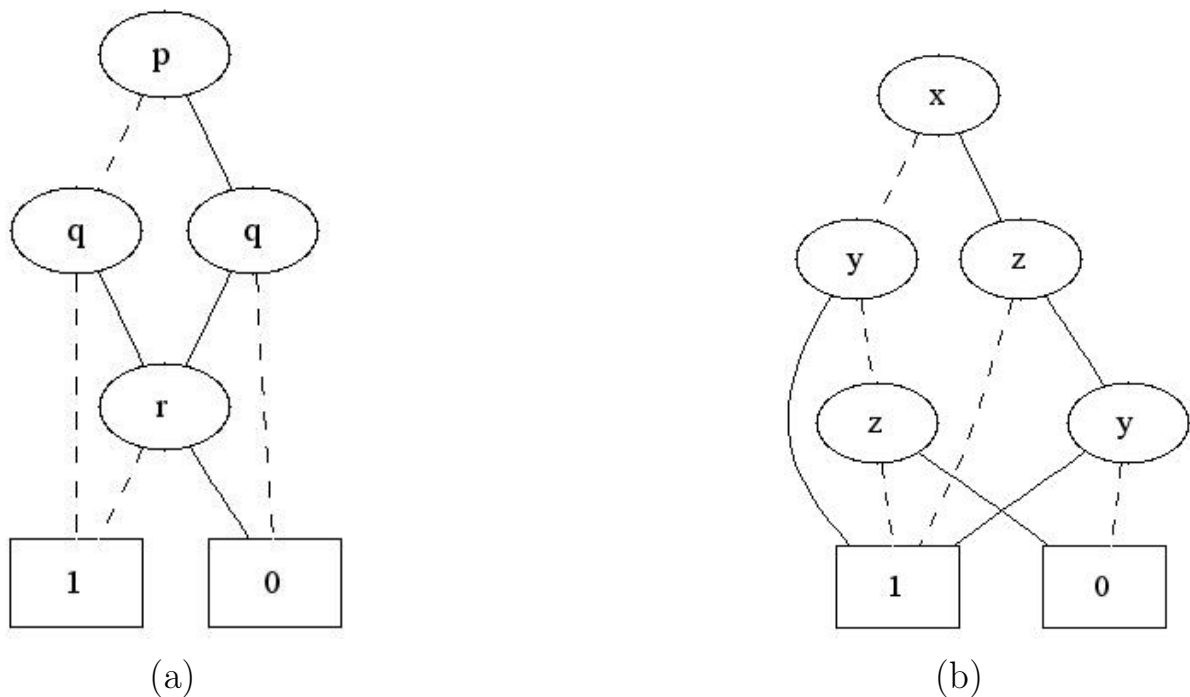


Рис. 1.2. Примеры BDD булевых функций

представление в форме BDD многих используемых на практике булевых функций растёт полиномиально или даже линейно.

Упорядоченная (ordered) BDD (OBDD) — это BDD, в которой переменные встречаются в фиксированном порядке. В частности, на рис. 1.2a переменные упорядочены $p < q < r$: вершина, помеченная p , находится в корне дерева, вершины q расположены ниже неё, а вершина r ещё ниже; иными словами для каждой переменной можно установить уровень, на котором расположены вершины, помеченные этой переменной и только ей. На рис. 1.2b продемонстрирована неупорядоченная BDD: по одному пути у нее порядок переменных $x < y < z$, а по другому $x < z < y$, соответственно, не возможно установить порядок переменных одинаковый для любого пути BDD.

Бинарная решающая диаграмма булевой функции может быть построена из ее семантического дерева применением операций редукции, которые состоят в следующем. Говорят, что «BDD — это семантическое дерево без избыточностей.» Правила редукции (алгоритма REDUCE):

1. Если несколько вершин орграфа помечено одним именем, и они имеют изоморфные подструктуры, то граф можно редуцировать: оставить только одну из этих подструктур. (Изоморфные подструктуры — это такие подструктуры, которые можно поставить

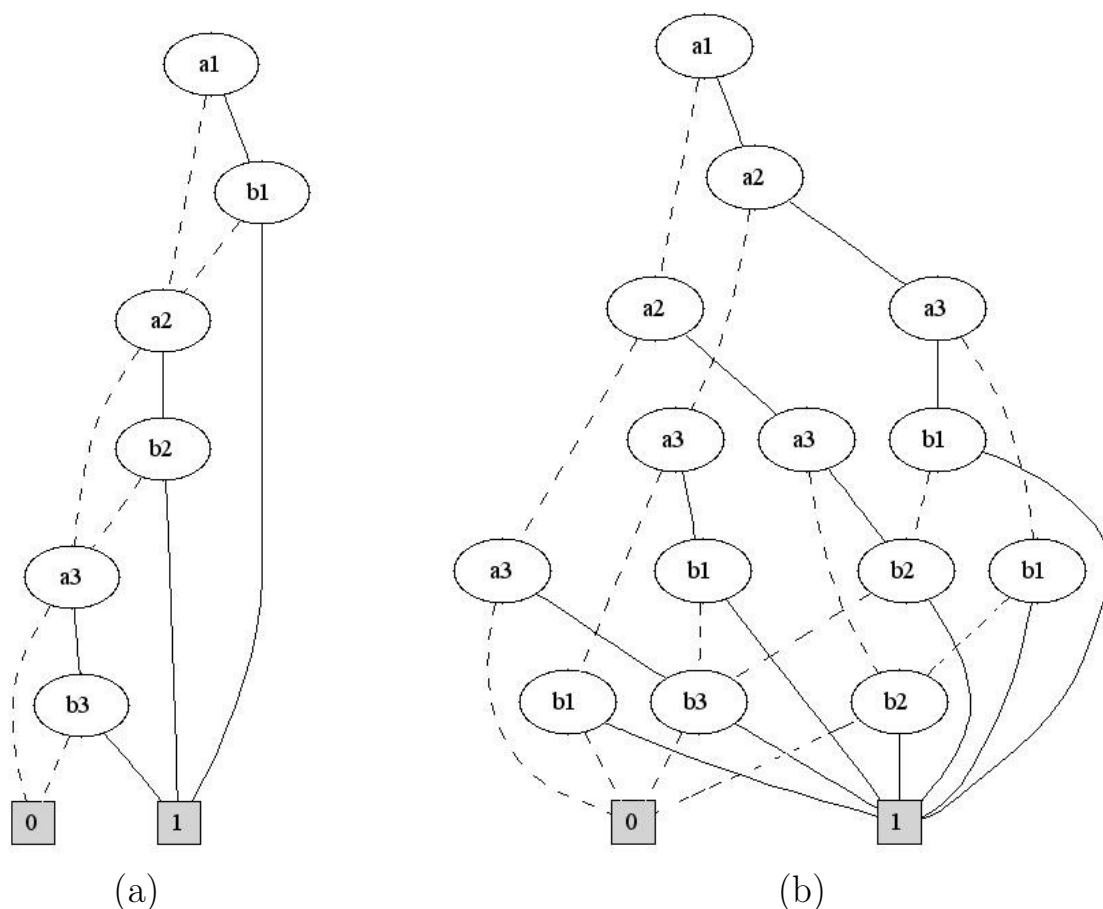


Рис. 1.3. Размер BDD булевой функции зависит от порядка переменных

в соответствие друг другу.)

2. Если обе выходные дуги промежуточной вершины v орграфа ведут в одну вершину, то эта вершина v может быть выброшена.

Упорядоченная BDD, полученная как результат многократного выполнения этих двух операций, является редуцированной (reduced). ROBDD — редуцированная упорядоченная бинарная решающая диаграмма — является каноническим представлением булевой функции. Далее будем под сокращением BDD подразумевать ROBDD.

Следует отметить, что размер BDD зависит от порядка переменных. На рис. 1.3а и 1.3б изображены представления функции $g = a_1 \wedge b_1 \vee a_2 \wedge b_2 \vee a_3 \wedge b_3$ при двух разных упорядочениях переменных. При порядке переменных $a_1 < b_1 < a_2 < b_2 < a_3 < b_3$ BDD функции g линейна от числа переменных (рис. 1.3а), при порядке $a_1 < a_2 < a_3 < b_1 < b_2 < b_3$ BDD растет экспоненциально от числа переменных (рис. 1.3б).

Задача нахождения оптимального порядка переменных при построении BDD в общем случае является NP-полной. Однако существуют

эффективные эвристики для нахождения субоптимального порядка.

По BDD можно легко построить ДНФ функции. Алгоритм построения ДНФ такой:

- по каждому пути, ведущему в терминальный узел 1, строим конъюнкцию. Выбираем путь. На этом пути если дуга, исходящая из вершины, пунктирная, то переменная войдет с отрицанием $\neg p$, а если сплошная, то переменная войдет без отрицания q ;
- полученные конъюнкции объединяем дизъюнкцией.

Например, по BDD на рис. 1.2а получается ДНФ булевой функции $f(p, q, r) = \neg p \wedge \neg q \vee \neg p \wedge q \wedge \neg r \vee p \wedge q \wedge \neg r$.

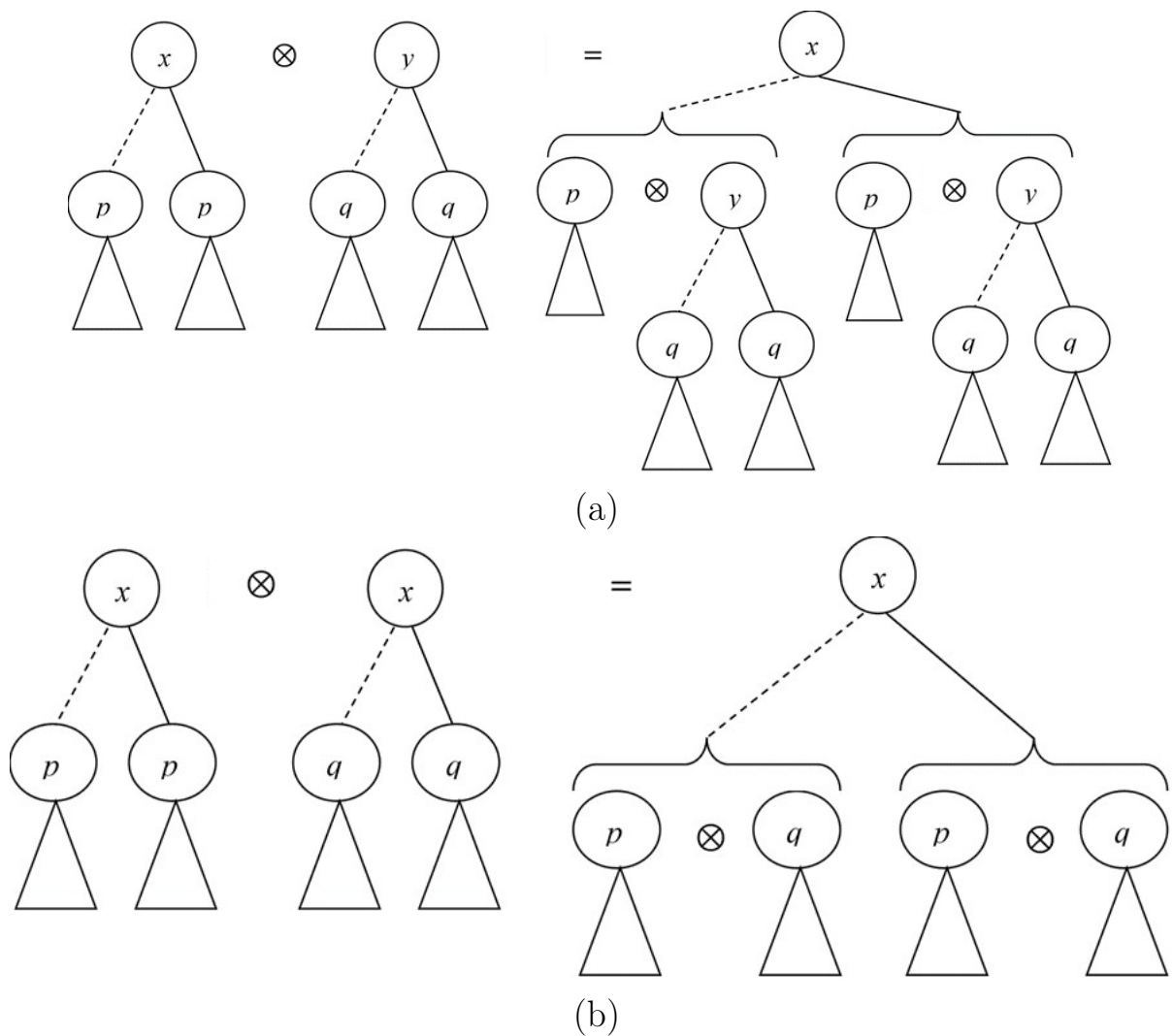


Рис. 1.4. Принципы алгоритма APPLY

Булевы операции над булевыми функциями, уже представленными в BDD, осуществляются с помощью алгоритма APPLY (рис. 1.4):

- Если переменная корневой вершины одной из BDD предшествует

по порядку переменной корневой вершиной другой BDD ($x < y$, рис. 1.4a), то вторая BDD дублируется в каждом подграфе первой BDD на уровень ниже, алгоритм повторяется;

- Если корневые вершины каждой BDD помечены одной переменной (x , рис. 1.4b), то каждая BDD расщепляется по штрихованной и сплошной ветке на два подграфа и алгоритм повторяется на уровень ниже;
- Когда алгоритм достигает листьев, то над листьями производится операция $\otimes$ в соответствии со свойствами этой операции для констант, результат записывается в лист.

Если в результате применения алгоритма APPLY появляется избыточность, то она устраняется с помощью алгоритма REDUCE, изложенного выше. Подробнее о BDD и их свойствах можно прочитать, например, [2].

2. РЕАЛИЗАЦИЯ BDD В БИБЛИОТЕКЕ BUDDY

BDD удобно представляется в компьютерной реализации. На рис. 2.1 приведено табличное представление BDD: каждой вершине графа отводится одна строка. В строке указываются номер вершины, номер переменной, и те две вершины, куда направлены два выхода, 0 и 1, из данной вершины.

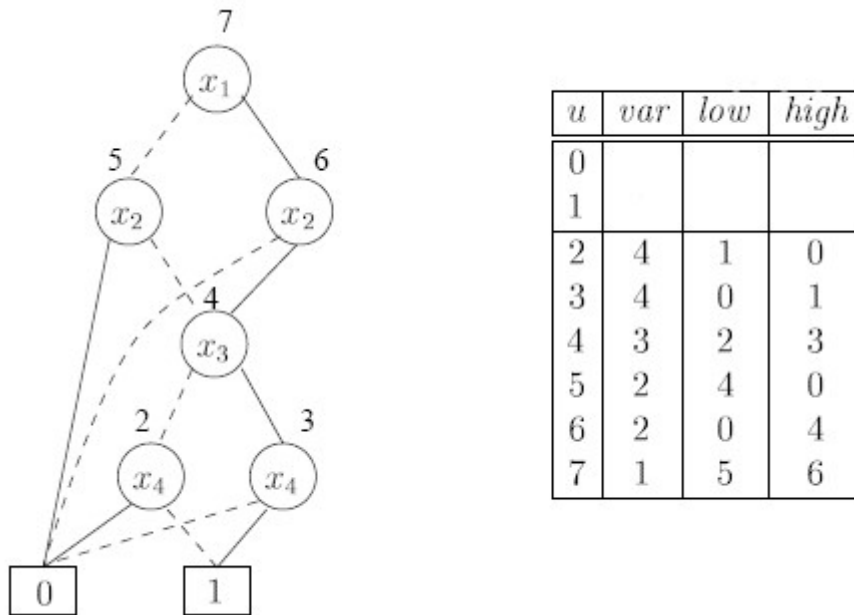


Рис. 2.1. Табличное представление BDD

Для представления BDD и выполнения операций над ними существует много различных библиотек. Мы рассмотрим свободно распространяемую библиотеку BuDDy для языка C++. В настоящий момент версия BuDDy 2.4 доступна по адресу:

<http://sourceforge.net/projects/buddy/>

В пакет BuDDy входят исходный код библиотеки, документация и примеры решённых задач. Для использования библиотеки необходимо подключить скомпилированный файл `bdd.lib` к проекту. Объявления типов данных и функций для работы с ними находятся в заголовочном файле `bdd.h`.

Основным типом данных в библиотеке BuDDy является класс `bdd`. Этот класс используется для задания BDD. Для этого класса перегружены следующие операторы:

<code>&</code>	<code>&=</code>	конъюнкция двух BDD (и соответственно с присваиванием)
<code> </code>	<code> =</code>	дизъюнкция
<code>∧</code>	<code>∧ =</code>	сложение по модулю 2
<code>!</code>		отрицание
<code>»</code>	<code>»=</code>	правосторонняя импликация
<code>«</code>	<code>«=</code>	левосторонняя импликация
<code>-</code>	<code>- =</code>	разность характеризуемых множеств: $a - b = a \wedge \neg b$, где a и b булевы функции
<code>></code>		булева функция «больше»
<code><</code>		булева функция «меньше»
<code>==</code>	<code>!=</code>	функции сравнения двух BDD. Возвращают значения «истина» или «ложь» в виде целых 0 или 1
<code>«</code>		оператор потокового вывода

Последний оператор может использоваться совместно с манипуляторами:

- `bddset` – вывод интерпретаций, на которых булева функция, представленная в форме BDD, истинна (этот режим по умолчанию);
- `bddtable` – вывод значимых строк таблицы (см. рис. 2.1);
- `bdddot` – вывод исходного кода программы `dot` пакета `graphviz`;
- `bddall` – вывод всех строк таблицы.

Для класса `bdd` определены константы `bddtrue` и `bddfalse`.

Начинать работу с библиотекой следует с вызова функции:

```
int bdd_init(int nodelist, int cachesize);
```

Она выделяет память для `nodelist` строк таблицы. При необходимости библиотека автоматически увеличит объём занимаемой памяти, на что, естественно, будет затрачено время.

В целях повышения быстродействия используется кэш для хранения результатов последних операций. Его статический размер устанавливается равным `cachesize`.

В документации к библиотеке рекомендуется использовать следующие значения описанных параметров:

- малые тестовые примеры: `nodelist = 1000, cachesize = 100`;
- задачи малой размерности: `nodelist = 10000, cachesize = 1000`;
- задачи средней размерности: `nodelist = 100000, cachesize = 10000`;
- задачи высокой размерности: `nodelist = 1000000, динамический`

кэш.

Для включения динамического кэша необходимо вызвать функцию

```
int bdd_setcacheratio(int cacheratio);
```

Здесь `cacheratio` – число строк таблицы, приходящееся на одно вхождение кэша.

После инициализации библиотеки пользователь должен установить число булевых переменных, используемых для решения задачи:

```
int bdd_setvarnum(int N);
```

Булева переменная представляется BDD вида рис. 2.2.

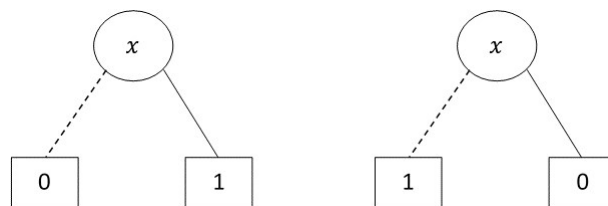


Рис. 2.2. BDD, задающие переменную x и $\neg x$

Для обращения к i -й переменной ($i \in 0..N - 1$) и её отрицанию необходимо воспользоваться следующими функциями соответственно:

```
bdd bdd_ithvar(int i);
bdd bdd_nithvar(int i);
```

Завершать работу с библиотекой следует вызовом функции

```
void bdd_done(void);
```

Рассмотрим выполнение различных операций над BDD на примере простой программы.

```
#pragma comment(lib, "bdd.lib") // подключение библиотеки
#include "bdd.h"                // включение заголовочного файла
#include <fstream>

using namespace std;

ofstream out;

void fun(char* varset, int size); // см. далее

void main(void)
{
    // Инициализация: начальное число вершин и статический кэш для
    // операций
    bdd_init(1000, 100);
```

```
bdd_setvarnum(6); // 6 булевых переменных
```

Возьмем для примера упомянутую ранее функцию $g = a_1 \wedge b_1 \vee a_2 \wedge b_2 \vee a_3 \wedge b_3$. Установим для неё следующий порядок переменных: $a_1 < a_2 < a_3 < b_1 < b_2 < b_3$.

```
bdd a1 = bdd_ithvar(0);  
bdd a2 = bdd_ithvar(1);  
bdd a3 = bdd_ithvar(2);  
bdd b1 = bdd_ithvar(3);  
bdd b2 = bdd_ithvar(4);  
bdd b3 = bdd_ithvar(5);
```

```
bdd g = a1&b1 | a2&b2 | a3&b3; // функция g(a1,a2,a3,b1,b2,b3)
```

Узнаем число вершин при таком порядке переменных при помощи функции `bdd_nodcount()`:

```
// число вершин BDD  
cout<<"\na1 < a2 < a3 < b1 < b2 < b3: "<<bdd_nodcount(g)  
<<" nodes\n "<<endl;
```

На экран будет выведен следующий результат:

```
a1 < a2 < a3 < b1 < b2 < b3: 14 nodes
```

Построим графически BDD функции g , используя программу `dot`.

```
out.open("bdd1.txt");  
out<<bdddot<<g<<endl; //вывод исходника dot  
out.close();
```

Результат обработки файла `bdd1.txt` на рис. 2.3а (слева).

Для построения BDD этой функции при порядке переменных $a_1 < b_1 < a_2 < b_2 < a_3 < b_3$ выполним следующие операции:

```
//меняем порядок переменных  
bdd_swapvar(1, 3);  
bdd_swapvar(2, 1);  
bdd_swapvar(2, 4);  
  
cout<<"\na1 < b1 < a2 < b2 < a3 < b3: "<<bdd_nodcount(g)  
<<" nodes"<<endl;  
out.open("bdd2.txt");  
out<<bdddot<<g<<endl;  
out.close();
```

На экран будет выведено

```
a1 < b1 < a2 < b2 < a3 < b3: 6 nodes
```

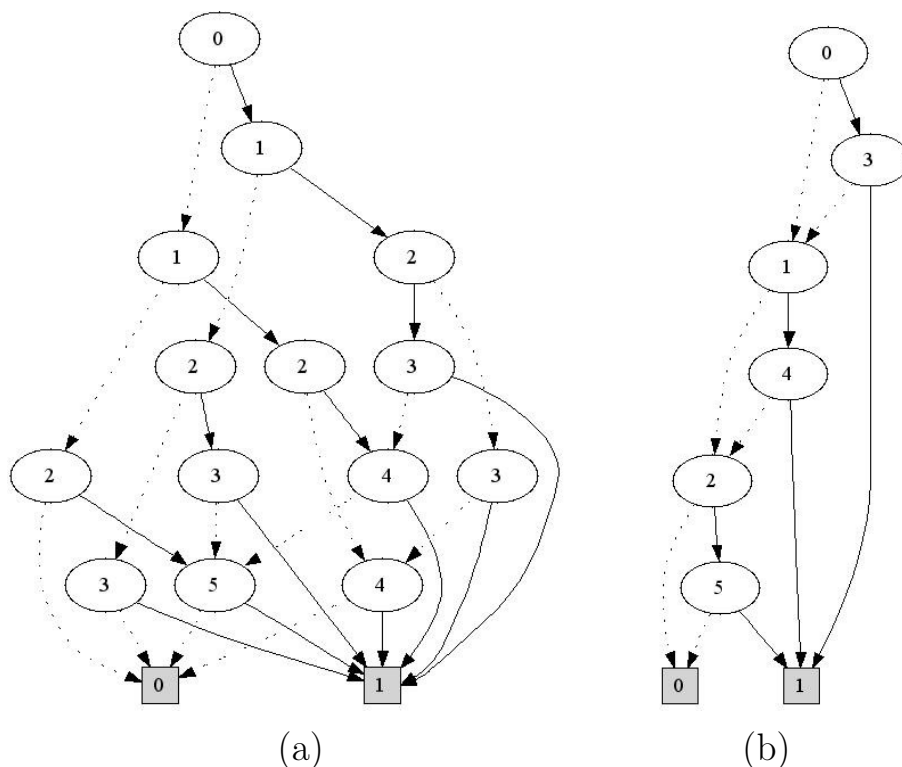


Рис. 2.3. BDD, вычисленные с помощью библиотеки BUDDy

Граф, закодированный в файле `bdd2.txt`, представлен на рис. 2.3b. Таким образом мы построили BDD булевой функции g при разном порядке переменных.

Выведем теперь все интерпретации, на которых функция g истинна, и количество этих интерпретаций (функция `bdd_satcount()`).

```
//вывод множеств, на которых g = 1
out.open("set.txt");
out<<bdd_satcount(g)<<" sats:\n"<<endl;
out<<bddset<<g<<endl;
```

Результат из файла `set.txt` выглядит так:

```
37 sats:

<0:0, 1:0, 2:1, 5:1>
<0:0, 1:1, 4:0, 2:1, 5:1>
<0:0, 1:1, 4:1>
<0:1, 3:0, 1:0, 2:1, 5:1>
<0:1, 3:0, 1:1, 4:0, 2:1, 5:1>
<0:1, 3:0, 1:1, 4:1>
<0:1, 3:1>
```

Функция истинна на 37 наборах. Интерпретации в наборах выводятся в формате 'номер переменной: значение'. Если переменная с номером

от 0 до N отсутствует, значит, она может быть равна как 0, так и 1 в совокупности с остальными переменными набора.

Часто возникает необходимость каким-либо образом обработать эти наборы. Для этого служит функция

```
void bdd_allsat(bdd r, bddallsathandler handler);
```

Здесь `handler` – указатель на функцию с прототипом `fun`

```
void fun(char* varset, int size);
```

Функция `bdd_allsat` для каждого такого набора выполняет функцию `fun`. Параметр `varset` – это массив элементов текущего набора. Если i -я переменная может быть равна как 0, так и 1, то соответствующий элемент массива будет отрицательным, `varset[i] = -1`. В параметре `size` указывается длина массива `varset`.

Пусть функция `fun` реализована следующим образом:

```
void fun(char* varset, int size)
{
    for (int i = 0; i < size; i++) out<<(varset[i] < 0 ? 'X'
    : (char)('0' + varset[i]));
    out<<endl;
}
```

Выведем теперь все интерпретации функции в файл `allsat.txt`:

```
out.open("allsat.txt");
out<<bdd_satcount(g)<<" sats:\n"<<endl;
bdd_allsat(g, fun);
out.close();
```

В файле `allsat.txt` получим результат:

```
37 sats:

001XX1
011X01
01XX1X
1010X1
111001
11X01X
1XX1XX
```

Чтобы получить BDD булевой функции, истинной только на одном из 37 интерпретаций, можно воспользоваться функцией

```
bdd bdd_satone(bdd);
```

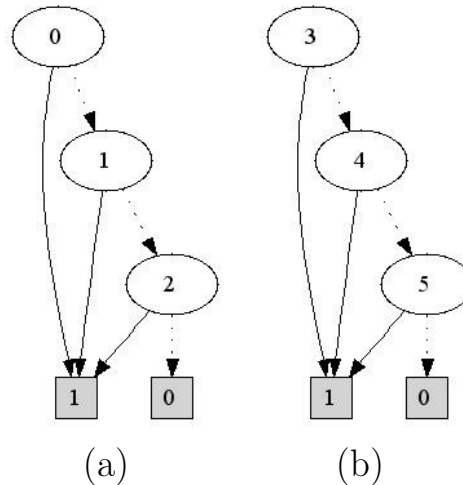


Рис. 2.4. Иллюстрации определения бинарных решающих диаграмм, представленных графом

При реализации неявных символьных алгоритмов когда булевы функции выступают в роли характеристических функций конечных множеств и бинарных отношений над ними, часто возникает необходимость исключить некоторые переменные из функции (например, при переходе от бинарного отношения к множеству). Формально это соответствует взятию квантора существования $\exists$ по соответствующей переменной: $\exists x.f(x, y)$. Т.к. переменная x принимает только два значения, то результат $\exists$ -квантификации соответствует выражению $\exists x.f(x, y) = f(0, y) \vee f(1, y)$.

Воспользуемся $\exists$ -квантификацией и исключим из функции $g = a_1 \wedge b_1 \vee a_2 \wedge b_2 \vee a_3 \wedge b_3$ переменные b_1, b_2 и b_3 .

```
//exist-квантификация по b1, b2 и b3
g = bdd_exist(g, b1&b2&b3);
```

```
out.open("bdd3.txt");
out<<bdddot<<g<<endl;
out.close();
```

Получившаяся BDD приведена на рис. 2.4а. Построив по BDD функцию g в ДНФ, получим $g = a_1 \vee a_2 \vee a_3$.

Заменим (переименуем) переменные a_1, a_2 и a_3 на b_1, b_2 и b_3 . Для этого воспользуемся следующим механизмом:

```
int A[] = {0, 1, 2}; // Будем менять переменные с номерами 0, 1,
2
int B[] = {3, 4, 5}; // на переменные с номерами 3, 4, 5
bddPair* pair = bdd_newpair(); // инициализируем множество пар
```

```

    bdd_setpairs(pair, A, B, 3);    // устанавливаем 3 пары: A[i]
    <-> B[i]
    bdd_freepair(pair); // уничтожаем пары

    out.open("bdd4.txt");
    out<<bdddot<<g<<endl;
    out.close();

```

Результирующая BDD представлена на рис. 2.4b.

Перед завершением работы выведем итоговую функцию $g = b_1 \vee b_2 \vee b_3$ в виде таблицы:

```

    out.open("Table.txt");
    out<<bddtable<<g<<endl;
    out.close();

    bdd_done(); // завершаем работу с библиотекой
}

```

Содержимое файла Table.txt будет таким:

```

ROOT: 23
[  12]   5 :   0   1
[  21]   4 :  12   1
[  23]   3 :  21   1

```

Интерпретировать результат нетрудно: коренная вершина имеет номер 23 и помечена переменной с номером 3, из 23 вершины 0-дуга идет в вершину с номером 21, а 1-дуга в лист 1 и т.д.

Нами были рассмотрены основные функции библиотеки BuDDy, необходимость в использовании которых возникает при решении большинства задач. Остальные функции библиотеки описаны в документации [3].

3. ЛОГИЧЕСКИЕ И ИХ РЕШЕНИЕ С ПОМОЩЬЮ БУЛЕВЫХ ФУНКЦИЙ

Рассмотрим логическую головоломку, известную как «Задача Эйнштейна».

5 разных человек в 5 разных домах разного цвета, курят 5 разных марок сигарет, выращивают 5 разных видов животных, пьют 5 разных видов напитков. Известно следующее:

- 1. Норвежец живет в первом доме.*
- 2. Англичанин живет в красном доме.*
- 3. Зеленый дом находится слева от белого.*
- 4. Датчанин пьет чай.*
- 5. Тот, кто курит Rothmans, живет рядом с тем, кто выращивает кошек.*
- 6. Тот, кто живет в желтом доме, курит Dunhill.*
- 7. Немец курит Marlboro.*
- 8. Тот, кто живет в центре, пьет молоко.*
- 9. Сосед того, кто курит Rothmans, пьет воду.*
- 10. Тот, кто курит Pall Mall, выращивает птиц.*
- 11. Швед выращивает собак.*
- 12. Норвежец живет рядом с синим домом.*
- 13. Тот, кто выращивает лошадей, живет в синем доме.*
- 14. Тот, кто курит Philip Morris, пьет пиво.*
- 15. В зеленом доме пьют кофе.*

Вопрос: Кто выращивает рыбок?

Очевидно, что для ответа на этот вопрос необходимо с каждым человеком сопоставить его национальность, марку сигарет, животное, напиток, цвет дома.

Мы хотели бы решить эту задачу с помощью булевых функций, представив условие задачи как набор логических ограничений.

Формализуем задачу.

Пусть имеется система, состоящая из N объектов, обладающих M свойствами. В качестве объектов выберем здесь номера домов, т. о. $N = 5$, Свойствами будут цвет дома, национальность проживающего и т. д. , $M = 5$. У двух различных объектов значения каждого свойства

не совпадают, т. о. каждое свойство может принимать N различных значений (см. табл. 3.1).

– Таблица 3.1 Свойства и их значения для задачи Эйнштейна

Номер	Название	Значения
1	цвет дома	красный, белый, зеленый, желтый, синий
2	национальность	англичанин, швед, датчанин, норвежец, немец
3	напиток	чай, молоко, пиво, вода, кофе
4	домашнее животное	кошка, лошадь, собака, птица, рыбка
5	сигареты	Rothmans, Dunhill, Marlboro, Pall Mall, Philip Morris

Поскольку множества значений свойств конечны, их можно закодировать с помощью двоичных переменных. Пусть j — номер элемента множества значений некоторого свойства ($j \in 0 \dots N - 1$). Для кодирования каждого номера элемента множества достаточно $\lceil \log_2(N) \rceil$ двоичных переменных. Так как $N = 5$, то для кодирования элементов множества свойства достаточно 3-х переменных, например, z_0, z_1, z_2 . В табл. 3.2 приведены название элементов множества свойства 1 (цвет), их десятичные номера и символьные коды, представленные с помощью двоичных переменных:

– Таблица 3.2 Кодирование элементов цвета

Элементы	Номер	Код
красный	0	$\neg z_0 \wedge \neg z_1 \wedge \neg z_2$
белый	1	$z_0 \wedge \neg z_1 \wedge \neg z_2$
зеленый	2	$\neg z_0 \wedge z_1 \wedge \neg z_2$
желтый	3	$z_0 \wedge z_1 \wedge \neg z_2$
синий	4	$\neg z_0 \wedge \neg z_1 \wedge z_2$

В таблице 3.3 приведены всевозможные значения свойств всех объектов задачи Эйнштейна в двоичном коде. Кодирование всех элементов потребует 75 двоичных переменных ($z_0 \dots z_{74}$).

Мы хотели бы, воспользовавшись условием задачи, булеву функцию F от 75 переменных, которая будет истинна на тех интерпретациях (двоичных наборах) из возможных 2^{75} , которые удовлетворяют ограничениям. Если задача имеет единственное решение, то функция F истинна

– Таблица 3.3 Кодирование элементов цвета

Объект 1					Объект 2					... Объект 5	
Свойство 1	Свойство 2	Свойство 3	Свойство 4	Свойство 5	Свойство 1	Свойство 2	Свойство 3	Свойство 4	Свойство 5	...	Свойство 4
000	000	000	000	000	000	000	000	000	000	...	000
001	001	001	001	001	001	001	001	001	001	...	001
...											
111	111	111	111	111	111	111	111	111	111	...	111

на одном единственном наборе двоичных значений ее аргументов. Если задача имеет несколько решений, то таких интерпретаций, на которых F истинна, будет несколько. В случае противоречивых условий задача может не иметь решений совсем. В этом случае функция F будет невыполнима ($F = false$).

Для построения функции F нам необходимо закодировать пространства всевозможных значений свойств из табл. 3.3. Когда пространство всевозможных интерпретаций небольшое, то кодирование обычно выполняется вручную. Здесь пространство интерпретаций 2^{75} , поэтому важно придумать некоторый алгоритм кодирования через переменные z , который впоследствии можно будет запрограммировать.

Для этого по табл. 3.3 упорядочим переменные z в трехмерный массив. Каждый элемент $z[i, k, t]$ массива является двоичной переменной. Индекс i соответствует номеру объекта и изменяется в диапазоне от 0 до $N - 1$, индекс k соответствует номеру свойства и изменяется в диапазоне от 0 до $M - 1$, а индекс t соответствует биту значения свойства и изменяется в диапазоне от 0 до T , где $\lceil \log_2(N) \rceil = T$.

Чтобы было удобнее работать с двоичными переменными, переведем эквивалентным образом трехмерный массив z в одномерный. Элементу $z[i, j, t]$ трехмерного массива будет соответствовать элемент $x[T \cdot M \cdot i + T \cdot k + t]$ одномерного массива.

Теперь с помощью элементов одномерного массива x построим функцию $p(k, i, j)$, описывающую, что свойство k объекта i имеет десятичное значение j , аналогично тому, как делали в табл. 3.2.

$$p(k, i, j) = \bigwedge_{t=0}^T y_{T \cdot M \cdot i + T \cdot k + t} =$$

где $y_{T \cdot M \cdot i + T \cdot k + t}$ — литерал переменной $x_{T \cdot M \cdot i + T \cdot k + t}$

$$y_{T \cdot M \cdot i + T \cdot k + t} = \begin{cases} x_{T \cdot M \cdot i + T \cdot k + t}, & \text{если } t\text{-й бит } j \text{ равен } 1; \\ \neg x_{T \cdot M \cdot i + T \cdot k + t}, & \text{если } t\text{-й бит } j \text{ равен } 0. \end{cases} \quad (3.1)$$

Алгоритм кодирования $p(k, i, j)$ таков. Зная номер элемента в множестве j , можно получить его код p при помощи младшего бита после оператора правого сдвига. Например, $p(k, i, j)$ для $j = 4$ вычисляется так:

$t = 0$	$(j \gg 0) \& 1 = 0$	$p(k, i, j) = \neg x_{T \cdot M \cdot i + T \cdot k}$
$t = 1$	$(j \gg 1) \& 1 = 0$	$p(k, i, j) \wedge = \neg x_{T \cdot M \cdot i + T \cdot k + 1}$
$t = 2$	$(j \gg 2) \& 1 = 1$	$p(k, i, j) \wedge = x_{T \cdot M \cdot i + T \cdot k + 2}$

В результате при $j = 4$ код будет $p(k, i, j) = \neg x_{T \cdot M \cdot i + T \cdot k} \wedge \neg x_{T \cdot M \cdot i + T \cdot k + 1} \wedge x_{T \cdot M \cdot i + T \cdot k + 2}$.

Теперь формализуем типы ограничений по характерным фразам из условия задачи, будем сохранять их в функции F :

1. Ограничение вида «*Норвежец живет в первом доме*» устанавливает, что свойство $k1$ конкретного объекта $i1$ имеет значение $j1$:

$$F := F \wedge p(k1, i1, j1).$$

Очевидно, что формализовав конкретную фразу, можно аналогичным образом формализовать любую другую фразу, соответствующую данной по смыслу.

2. Ограничение вида «*Англичанин живет в красном доме*» устанавливает соответствие между двумя свойствами какого-либо объекта:

$$F := F \wedge (p(k1, i, j1) \leftrightarrow p(k2, i, j2)), \text{ для всех } i \text{ от } 0 \text{ до } N - 1.$$

3. (а) Ограничение расположения объектов – расположение «слева» («справа»). Это ограничения типа объект, у которого свойство $k1$ имеет значение $j1$, расположен слева от объекта, у которого свойство $k2$ имеет значение $j2$ («*Зеленый дом находится слева от белого*»):

$$F := F \wedge (p(k1, i, j1) \leftrightarrow p(k2, i+1, j2)), \text{ для всех } i \text{ от } 0 \text{ до } N - 2. \quad (3.2)$$

- (b) К формуле (3.2) нужно добавить ограничение: объект, у которого свойство $k2$ имеет значение $j2$, не будет первым, а объект, у которого свойство $k1$ имеет значение $j1$, не будет последним.

$$F := F \wedge \neg p(k2, 0, j2) \wedge \neg p(k1, N - 1, j1).$$

4. Ограничение расположения объектов – расположение «рядом» – дизъюнкция ограничений «слева» и «справа» типа 3.

По умолчанию в задаче Эйнштейна подразумеваются еще следующие ограничения:

5. У двух различных объектов значения любого параметра (свойства) не совпадают.

$$F := F \wedge (p(k, i1, j) \rightarrow \neg p(k, i2, j)) \text{ при } i1 \neq i2.$$

6. Параметры принимают значения только из заданных множеств (значение свойств должно быть меньше N).

Изначально $F = true$. Введя в любом порядке имеющиеся ограничения, получим искомую функцию F .

4. КУРСОВАЯ РАБОТА «ИСПОЛЬЗОВАНИЕ БИНАРНЫХ РЕШАЮЩИХ ДИАГРАММ ДЛЯ РЕШЕНИЯ ЛОГИЧЕСКИХ ЗАДАЧ»

Курсовая работа построена на основе задачи Эйнштейна параметризацией условия. Сначала рассмотрим упрощенный вариант такой параметризации с решением в BUDDy.

4.1. ПРИМЕР РЕШЕНИЯ УПРОЩЕННОГО ЗАДАНИЯ КУРСОВОЙ РАБОТЫ

Условие:

- выбрать систему, состоящую из N объектов, которые обладают M свойствами, принимающими N различных значений;
- задать $n1$ ограничений типа 1, $n2$ ограничений типа 2, $n3$ ограничений типа 3 и $n4$ ограничений типа 4 (типы ограничений возьмите из задачи Эйнштейна, $n_i > 0$);
- найти все возможные решения;
- придумать физическую интерпретацию задачи.

Для выполнения задания необходимо написать программу на языке C++ с использованием библиотеки BuDDy.

Решение задачи

Пусть значения параметров задачи таковы.

$N = 3, M = 3, n1 = 1, n2 = 1, n3 = 1, n4 = 1.$

Для кодирования трёх разных значений параметров требуются две булевы переменные. Следовательно, для описания всего пространства решений необходимо 18 переменных.

Приведем программу для предложенной задачи.

```
#pragma comment(lib, "bdd.lib")
#include "bdd.h"
#include <fstream>

using namespace std;

#define N_VAR 18 // число булевых переменных
#define N 3     // число объектов
#define M 3     // число свойств
#define LOG_N 2
```

```

ofstream out;

void fun(char* varset, int size); // функция, используемая для
    вывода решений

void main(void)
{
    // инициализация
    bdd_init(10000, 1000);
    bdd_setvarnum(N_VAR);

    // --> вводим функцию p(k, i, j) следующим образом ( кодируем
    pk[i][j] ):
    bdd p1[N][N];
    bdd p2[N][N];
    bdd p3[N][N];

    unsigned I = 0;
    for (unsigned i = 0; i < N; i++)
    {
        for (unsigned j = 0; j < N; j++)
        {
            p1[i][j] = bddtrue;
            for (unsigned k = 0; k < LOG_N; k++) p1[i][j] &= ((j >>
k) & 1) ? bdd_ithvar(I + k) : bdd_nithvar(I + k) ;
            p2[i][j] = bddtrue;
            for (unsigned k = 0; k < LOG_N; k++) p2[i][j] &= ((j >>
k) & 1) ? bdd_ithvar(I + LOG_N + k) : bdd_nithvar(I +
LOG_N + k) ;
            p3[i][j] = bddtrue;
            for (unsigned k = 0; k < LOG_N; k++) p3[i][j] &= ((j >>
k) & 1) ? bdd_ithvar(I + LOG_N*2 + k) : bdd_nithvar(I +
LOG_N*2 + k) ;
        }
        I += LOG_N*M;
    }

    // здесь должны быть ограничения
    // они описаны формально при рассмотрении задачи Эйнштейна

    // вывод результатов
    out.open("out.txt");
    unsigned satcount = (unsigned)bdd_satcount(task);
    out<<satcount<<" solutions:\n"<<endl;
    if (satcount) bdd_allsat(task, fun);
    out.close();
}

```

```

    bdd_done(); // завершение работы библиотеки
}

// Ниже приведена реализация функций, управляющих выводом результатов.
// Рекомендуется самостоятельно с ними ознакомиться.
// В собственных заданиях следует использовать эти функции
// или придумать собственные.

char var[N_VAR];

void print(void)
{
    for (unsigned i = 0; i < N; i++)
    {
        out<<i<<" : ";
        for (unsigned j = 0; j < N; j++)
        {
            unsigned J = i*N*LOG_N + j*LOG_N;
            unsigned num = 0;
            for (unsigned k = 0; k < LOG_N; k++) num += (unsigned)(
var[J + k] << k);
            out<<num<<' ';
        }
        out<<endl;
    }
    out<<endl;
}

void build(char* varset, unsigned n, unsigned I)
{
    if (I == n - 1)
    {
        if (varset[I] >= 0)
        {
            var[I] = varset[I];
            print();
            return;
        }
        var[I] = 0;
        print();
        var[I] = 1;
        print();
        return;
    }
    if (varset[I] >= 0)
    {
        var[I] = varset[I];

```

```

        build(varset, n, I + 1);
        return;
    }
    var[I] = 0;
    build(varset, n, I + 1);
    var[I] = 1;
    build(varset, n, I + 1);
}

void fun(char* varset, int size)
{
    build(varset, size, 0);
}

```

Запускаем программу, получаем следующий ответ:

5 solutions:

```

0: 0 0 2  // у 0-го объекта: свойство 1 имеет значение 0,
1: 1 2 0  // свойство 2 - значение 0, свойство 3 - значение 2; у 1-го
2: 2 1 1  // объекта: свойство 1 имеет значение 1, и т.д.

```

```

0: 0 0 1
1: 1 2 0
2: 2 1 2

```

```

0: 0 0 1
1: 1 1 2
2: 2 2 0

```

```

0: 0 2 0
1: 2 0 2
2: 1 1 1

```

```

0: 0 1 1
1: 2 0 2
2: 1 2 0

```

Интерпретация решения

Придумаем интерпретацию решения на естественном языке.

Дедка, Жучка и внучка тянут репку. У всех разный цвет глаз (серые, зелёные, голубые). Известно, что герой либо капризный, либо имеет хвост, либо пенсионер.

У нас имеется три героя (номер в очереди к репке) и три свойства. Предлагается закодировать домены свойств как указано на табл. 4.1.

Рассмотрим первое решение из выведенных программой выше:

имя	дедка=0, Жучка=1, внучка=2
цвет глаз	сероглазый=0, зеленоглазый=1, голубоглазый=2
характер	капризный=0, хвостатый=1, пенсионер=2

– Таблица 4.1 Коды значений свойств в задаче «Репка»

```
0: 0 0 1
1: 1 1 2
2: 2 2 0
```

При интерпретации этого решения на естественном языке получим: *Дедка с серыми глазами и хвостом держится за репку. Зеленоглазый пенсионер Жучка держится за дедку. За Жучку держится капризная голубоглазая внучка.*

Как видно, ни одно из утверждений этого решения (и других найденных решений) не противоречит условиям задачи.

Оставим одно решение, добавив еще одно условие: *У внучки голубые глаза.*

Результат:

```
1 solutions:

0: 0 0 1
1: 1 1 2
2: 2 2 0
```

А сейчас предположим, что в условии задачи задано, что *Жучка тоже имеет хвост*. В результате не останется ни одного решения:

```
0 solutions:
```

Заметим, что решение подобных задач было не возможно в других представлениях булевых функций. BDD и их программная реализация позволяет решать задачи с таким большим количеством переменных.

4.2. УСЛОВИЕ КУРСОВОЙ РАБОТЫ

Для курсовой работы условие задачи из раздела 4.1 будет усложнено.

4.2.1. Изменение позиционирования объектов

Пусть имеется $N = 9$ объектов.

В задаче с репкой и в задаче Эйнштейна объекты следовали один за другим, образуя линию. В нашей задаче объекты занимают ячейки на квадрате (см. рис. 4.1а).

4.2.2. Введение соседских отношений

	1	2	3
1			*
2		0	
3			*

(a)

	1	2	3
		*	
0			
		*	

(b)

	1	2	3
0			
		*	

(c)

	1	2	3
		*	
0			

(d)

Рис. 4.1. «Соседские» отношения объектов

Введем понятие соседских отношений для объектов. Сначала зададим соседские отношения для центрального объекта. Так на рис. 4.1a соседями центрального объекта 0 являются объекты справа-сверху * и справа-снизу *. Т.о. ограничение типа 4 из задачи Эйнштейна, «Норвежец живет рядом с синим домом», для этого центрального устанавливается относительно соседей элемента (справа-сверху и справа-снизу). Аналогично понимаются и ограничения типа 3: «справа» в задаче Эйнштейна — это, например, «справа-сверху» на рис. 4.1a, а «слева» — это «справа-снизу».

Соседи других элементов строятся по рис. 4.1a как по шаблону. Сдвигаем шаблон так, чтобы 0 оказался на месте той ячейки, для которой необходимо найти соседей. Например, для объекта 0 на рис. 4.1b картинка будет схожа с рис. 4.1a, а для объекта рис. 4.1c останется только один сосед справа-снизу, и т.д. (рис. 4.1d). Для остальных объектов нетрудно вычислить соседей аналогично, в частности, у объектов в 3-м столбце не будет ни одного соседа.

4.2.3. Введение склейки

Другое изменение состоит во введении «склейки». Склейка — это изменение границ области с объектами, а, именно, склеивание границ. Это то же самое, что в задаче Эйнштейна домики выставить не в ряд, а по кругу. Когда объекты выставлены в ряд, то у крайне левого элемента «рядом» может быть только один сосед, когда объекты расположены по кругу, то у крайне левого элемента рядом оказывается крайний правый сосед. Склейку в нашем случае будем производить либо по столбцам (по верхней-нижней границам), либо по строкам (по левой-правой границам). Пусть для предыдущего примера есть склейка по столбцам, тогда у объекта в 1-м ряду и 1-м столбце появляется сосед справа-сверху (это элемент в 3-м ряду и 2-м столбце, рис. 4.2a), а у

объекта 3-м ряду и 1-м столбце появляется сосед справа-снизу (рис. 4.2b). Изменения для других элементов вычисляются аналогично (поменяются соседи у объекта из 1 ряда и 2 столбца и у объекта из 3 ряда и 2 столбца).

	1	2	3
1	0		
2		*	
3		*	

(a)

1	2	3
	*	
	*	
0		

(b)

Рис. 4.2. Соседские отношения объектов с рис. 4.1 со склейкой по первому столбцу

4.2.4. Творческая модификация

Введем дополнительное свойство, например, отражающее отношение между парами объектов задачи: «ненавидит», «уважает», «любит». Об этом свойстве предполагается следующее.

- Отношения могут быть не симметричными, то есть объект i может «любить» объект j , в то время, как объект j может «ненавидеть» объект i .
- Значение этого свойства не уникально для объектов – несколько объектов могут «испытывать» одинаковые чувства, то есть объект i может «любить» какой-то объект и объект j может «любить» какой-то объект.
- Отношение между объектами может быть неопределено, например, когда объект испытывает к другому объекту и «любовь», и «ненависть».

Введите кодирование, необходимое для решения этой задачи. Сформулируйте самостоятельно 4-5 ограничений в виде предикатов с этим свойством в виде формул матлогики. Найдите все решения задачи с помощью BDD.

Примеры ограничений с этим свойством представлены.

- Соседи не могут ненавидеть друг друга.
- Есть пары объектов, отношения между которыми взаимны.
- Не у всех объектов отношение друг к другу определено.
- Есть такой объект, который уважает одного из своих соседей, но любит другого соседа.

Задание сформулировано в свободной форме. Есть несколько

моментов, на которые надо обратить внимание.

- Студент может придумать любые отношения (необязательно про «любовь»), важно, чтобы отношения были между парами объектов, и количество возможных определенных значений этого отношения было 3 (здесь, «ненавидит», «уважает», «любит»). Наиболее естественно вводить отношения, согласующиеся с вашей интерпретацией.
- При составлении ограничений необходимо ввести несколько ограничений, связывающих отношение соседства и это свойство (например, «Есть такой объект, который ненавидит одного из своих соседей, но любит другого соседа»).
- При составлении ограничений, связанных с этим свойством, должны быть ограничения, использующие кванторы (например, «Есть такой объект, который ненавидит одного из своих соседей, но любит другого соседа»).
- Это свойство и ограничения, связанные с ним, необходимо встроить в решение задачи с уникальными свойствами. Если количество переменных при этом окажется слишком велико, то можно часть уникальных свойств и связанных с ними ограничений убрать.

4.2.5. Формулировка условия работы

Теперь сформулируем условие курсовой работы. Курсовая работа выполняется индивидуально, в соответствии со своим вариантом. Задача определяется следующими параметрами.

- Имеется $N = 9$ объектов с $M = 4$ свойствами. Свойства принимают N различных значений.
- Имеются индивидуальные соседские отношения, склейка, обязательное количество ограничений n_1, n_2, n_3, n_4 .
- Дополнительно определена дельта C (используется для вычисления варианта).

Курсовую работу можно выполнять на разных уровнях:

1. **простой** — без склейки;
2. **продвинутый** — выполняется вариант без склейки и со склейкой;
3. **сложный** — то же, что на продвинутом, плюс творческая модификация.

В отчете необходимо объяснить результаты. В зависимости от выбранного вами уровня выполнения и качества исполнения, оценивается курсовая работа. Количество баллов в зависимости от уровня сложности

выполнения курсовой работы приведено в Moodle.

Опишем последовательность действий при варианте без склейки:

1. Определите соседей всех объектов по своему варианту без склейки.
2. Модифицируйте ограничения типа 3 и 4 задачи Эйнштейна с учетом соседских отношений вашего варианта. Задайте n_3 ограничений подобных отношениям типа 3 и n_4 ограничений подобных отношениям типа 4 в соответствии со своим заданием.
3. Придумайте n_1 ограничений 1-ого типа и n_2 ограничений 2-ого типа.
4. Постройте программу для решения вашей задачи с использованием BDD.
5. Найдите все возможные решения.
6. Придумайте физическую интерпретацию задачи.
7. Выпишите интерпретацию ограничений на естественном языке.
8. Оставьте около 12-16 решений задачи. Этого можно добиться, добавив дополнительные ограничения любого типа или модифицировав текущие. Если у вас получилось меньше решений, то часть ограничений типа 1-2 можно убрать.
9. Выберите одно из решений и запишите его интерпретацию на естественном языке.

Если вы выбираете продвинутый уровень, то повторяете предыдущую последовательность для случая, когда склейка есть. Интерпретация сохраняется одинаковой для всех уровней.

Требования к реализации, если вы сдаете на компьютере лаборатории: реализация на языке C++ с использованием библиотеки BuDDy. Компилировать под MS Visual Studio.

Для выполнения задания необходимо написать программу. Реализация должна быть масштабируемой, т. е. изменение параметров N , M не должны приводить к изменению основной части программы.

4.3. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ДЛЯ КУРСОВОЙ РАБОТЫ

Найдите свой номер в списке группы из файла аттестации — пусть это будет A . Определите пять последних цифр номера вашей группы — B . Дельта C выдана вам дополнительно.

Вычислите следующие числа:

$$\begin{aligned}a &= A + B \\b &= A + C \\v1 &= (a \bmod 28) + 1 \\v2 &= (b \bmod 2) + 1 \\v3 &= (A \bmod 4) + 1\end{aligned}$$

Пусть $A = 5$, $B = 90101$, $C = 3$, тогда $v1 = 3$, $v2 = 3$, $v3 = 2$.

Числа $(v1, v2, v3)$ задают ваш вариант. По этим числам выбираете соответствующие варианты из таблиц, приведенных ниже: $v1$ — задает соседские отношения (табл. 4.4), $v2$ — склейку (табл. 4.2), $v3$ — количество ограничений по типам (табл. 4.3).

– Таблица 4.2 Варианты склейки

$v2$	1	2
	левая-правая границы	верхняя-нижняя границы

– Таблица 4.3 Варианты с количествами ограничений

$v3$	n1	n2	n3	n4
1	6	8	5	3
2	7	4	2	6
3	7	5	5	4
4	4	6	3	5

4.4. ТРЕБОВАНИЯ К ВЫПОЛНЕНИЮ КУРСОВОЙ РАБОТЫ

Работа должна быть оформлена по требованиям СПбПУ [6].

Курсовая работа выполняется студентом лично.

В результате выполнения работы студент должен знать основные понятия, связанные с BDD, понимать основные алгоритмы построения BDD, уметь программно решать логические задачи с использованием BDD.

– Таблица 4.4 Варианты шаблонов для соседских отношений

$v1$				$v1$				$v1$				$v1$			
1	*	*		2		*	*	3			*	4			
		0				0				0	*		*	0	
															*
5	*		*	6		*		7			*	8			
		0			*	0				0				0	*
									*				*		
9	*			10		*		11			*	12			
	*	0				0	*			0				0	*
									*				*		
13	*			14		*		15			*	16			
		0	*			0				0				0	*
					*						*				*
17	*			18		*		19				20			
		0				0			*	0	*			0	
	*					*							*	*	
21	*			22		*		23				24			
		0				0			*	0				0	
			*				*		*				*		*
25	*			26			*	27				28			
		0			*	0			*	0				0	
			*							*			*	*	

ЗАКЛЮЧЕНИЕ

В пособии изложена курсовая работа на основе модифицированной задачи Эйнштейна.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Карпов Ю.Г. Теория автоматов. СПб.:Питер, 2013, 208 стр.
2. Карпов Ю.Г. Model Checking. Верификация параллельных и распределенных программных систем. СПб:БХВ-Петербург, 2010, 560 стр.
3. Lind-Nielsen, J. BuDDy: Binary Decision Diagram package. IT-University of Copenhagen, 2002
4. Математическая логика. СПбПУ. Электронный ресурс. Платформа openedu.ru, <https://openedu.ru/course/spbstu/MATLOG/> дата доступа 28.09.2021
5. Математическая логика. Политехнический взгляд. СПбПУ. Электронный ресурс. Платформа Coursera. <https://ru.coursera.org/learn/matematicheskaya-logika-politekhnikheskiy-vzglyad> дата доступа 28.09.2021
6. Регламент содержания, оформления, организации выполнения и защиты курсовых проектов и курсовых работ. СПбПУ. 2018. электронный ресурс <https://www.spbstu.ru/upload/dmo/course-regulations.pdf> дата доступа 28.09.2021

Шошмина Ирина Владимировна

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ К КУРСОВОЙ РАБОТЕ ПО
МАТЕМАТИЧЕСКОЙ ЛОГИКЕ «ИСПОЛЬЗОВАНИЕ БИНАРНЫХ
РЕШАЮЩИХ ДИАГРАММ ДЛЯ РЕШЕНИЯ ЛОГИЧЕСКИХ
ЗАДАЧ» (ЧЕРНОВИК)

Учебное пособие

Лицензия ЛР № 020593 от 07.08.97

Налоговая льгота — Общероссийский классификатор продукции
ОК 005-93, т.2; 95 3005 — учебная литература

Подписано в печать 2024. Формат 60x84/16. Печать цифровая
Усл. печ. л. . Уч.-изд. л. . Тираж 40. Заказ .

Отпечатано с готового оригинал-макета, предоставленного авторами,
в Цифровом типографском центре
Издательства Политехнического университета,
195251, Санкт-Петербург, Политехническая ул., 29.
Тел.: (812) 550-40-14.
Тел./факс: (812) 297-57-76.